Appendix for BitEngine

A Operator Details

In this section, we list operator details that have not appeared in the BitEngine manuscript due to space constraints.

A.1 Projection

The Projection operator retrieves the columns specified in the SELECT clause and evaluates element-wise expressions on the qualifying tuples (e.g., multiplication in our example).

Baseline DBMS typically employs one of the two approaches to retrieve projection columns. (1) It performs selection on predicate columns (*l_quantity* and *l_shipdate* in our example), before selectively retrieving projection columns (*l_price* and *l_discount*). (2) It retrieves both predicate and projection columns simultaneously, utilizing predicate pushdown (e.g., zone maps) to skip Data Chunks prior to selection. While the first approach minimizes I/Os and fits pure columnar stores (e.g., MonetDB) for highly selective queries, the second approach maximizes sequential I/Os for modern analytical DBMSs that leverage Parquet-based storage and vectorized engines. Our evaluation confirms that the second approach outperforms the first once query selectivity exceeds 1% in our testbed; we therefore adopt the second approach as our projection baseline. Consequently, Data Chunks transferred across operators contain both predicate and projection columns, along with qualifying RIDs stored in SELs. Using these Data Chunks, the baseline iterates over the qualifying tuples and evaluates the projection expression (multiplication) element-wise with scalar instructions.

BitEngine performs selection via bitmap indexing, eliminating the need to scan predicate columns. Unlike the baseline, which relies on zone maps, BitEngine retrieves precisely the qualifying segments of the projection columns irrespective of workload statistics, a critical advantage in real workloads (in particular, those with uniform distributions). Furthermore, the selection result bitvector serves as a mask, letting BitEngine naturally apply masked SIMD operations to these segments for the projection.

A.2 Converting between RID and Bitvector

```
// Convert RID list to bitvector
convert_RID_to_btv(RIDs[], btv[])
  for (each ID in RIDs[]):
    word = ID >> 6 // Find the matching 64-bit word
    mask = 1 << (ID & 0x3F) // Offset in the word
    btv[word] |= mask

// Convert bitvector to RID list
convert_btv_to_RID(btv[])
  base_index := 0
  for (each 32-bit batch in btv[]):
    if (popcnt(batch) == 0): continue
    // Extract the IDs corresponding to the 1s in batch
    tmp[] = _mm512_maskz_compress(bits, [0, 1, 2, ..., 15])
    // Add starting index to qualifying IDs
    RID[] += _mm512_add_epi32(tmp[], base_idx)
    base_index += 16
  return RID[]
```

Algorithm 1: Converting RIDs to bitvector, and vice versa.

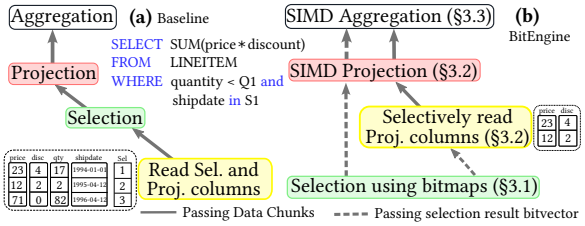

Figure 1: In columnar DBMSs using Parquet-based storage, (a) the baseline reads all predicate and projection columns, then performs selection, projection, and aggregation operations on RIDs stored in each Data Chunk's SEL. In contrast, (2) BitEngine first retrieves selection result bitvectors using bitmap indexing, enabling selective retrieval of projection columns, SIMD projection and aggregation operations.

Our findings in §3.2 raises a fundamental question: *Can DBMSs efficiently convert between a RID list and a bitvector when necessary?* To answer this, we implement both conversion algorithms. As illustrated in Algorithm 1, converting RID lists to bitvectors is mainly scalar-based. Applying SIMD to this function is complex and significantly degrades performance because contiguous RIDs often map to different memory words (Alg. 3, Lines 4–6), thus breaking SIMD instruction efficiency. Our evaluation demonstrates that the proposed scalar version achieves the best performance. In contrast, the reverse conversion fully leverages AVX-512 instructions. It extracts qualifying RIDs from set bits in the bitvector and adds the appropriate starting index to form the final RIDs (Lines 14–16), in 16-value batches.

B TPC-H Queries

In the BitEngine manuscript, we discussed how BitEngine executes TPC-H Q1 (grouping), Q5 (join), and Q6 (scan) in details. In this appendix, we present two additional representative TPC-H queries (Q10 and Q17) that could not fit in the BitEngine manuscript. The key points are *how BitEngine efficiently executes joins in SPJA queries using bitmap indexing and why downstream operators, such as post-join grouping and late materialization, remain lightweight.*

B.1 Q10

In SPJA joins, the bottleneck is the fact-dimension join rather than grouping. We use Q10 as a running example (with its query plans in Figure 2) to demonstrate this.

Baseline first chain-joins ORDERS with CUSTOMER (and NATION) using DuckDB's standard hash join, producing an intermediate table of qualifying order keys together with the associated customer attributes. This stage is lightweight, taking only 120ms. For the downstream LINEITEM-ORDERS join under late materialization, DuckDB scans the LINEITEM table and applies the predicate *l_returnflag* = 'R', taking 490ms. DuckDB next builds a hash table on these LINEITEM rows and probes it with the order keys from the ORDERS-CUSTOMER side (470ms), yielding an intermediate

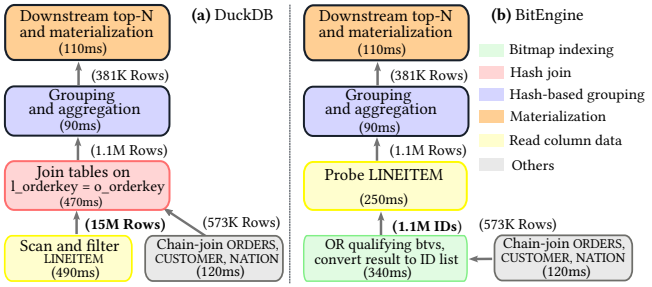

Figure 2: Query plans of TPC-H Q10 (SF = 10) on (a) DuckDB and (b) BitEngine, with each operator annotated by its execution time and the number of rows it passes downstream. DuckDB spends most of its time on the fact-table scan and hash join, whereas BitEngine avoids this bottleneck using bitmap indexing. Meanwhile, the downstream operators, in particular grouping, are lightweight because they process only the qualifying rows that pass the join predicate, so BitEngine reuses the standard operators for them. Figure 3 breaks down the resulting latencies.

table of 1.1M rows (at SF=10, 2% of the original table). On this much smaller intermediate table, grouping and aggregation (90ms) and top-20 selection with late materialization (110ms) are both fast.

BitEngine chain-joins ORDERS with CUSTOMER and NATION, as in the baseline, producing qualifying order keys with their associated (*c_custkey*, *n_nationkey*) pairs. For each qualifying order key, BitEngine fetches the corresponding bitvector from the *l_orderkey* bitmap and ORs it into a single result bitvector *btv_res*, while recording the mapping from each order key to its (*c_custkey*, *n_nationkey*) in a hash map. BitEngine then ANDs *btv_res* with the *l_returnflag* = 'R' bitvector and converts the result to an ID list of all qualifying LINEITEM rows. This stage takes 340ms. BitEngine next uses the ID list to probe LINEITEM once. For Q10, this fetches only 1.1M rows, 2% of what the baseline reads, and takes only 250ms. BitEngine aggregates the fetched rows, grouped by (*c_custkey*, *n_nationkey*) via the order-key map, and selects the top-20 customers. Finally, it late-materializes the remaining customer and nation columns (*c_name*, *n_name*) for the 20 winners only.

Overall, **SPJA queries are commonly dominated by the fact-dimension join**. In Q10, the LINEITEM scan and the hash-table build/probe together account for 75% of end-to-end latency. BitEngine (a) replaces the hash-table build/probe with bitmap indexing and (b) avoids the full fact-table scan, probing only the qualifying rows that pass the join predicate. The combined Indexing + Probing time of BitEngine is 590ms, 1.6× faster than the baseline's 960ms hash join. On the other hand, because such joins are highly selective (e.g., 0.1% for Q5, 2% for Q10, and 0.1% for Q17), the intermediate table is small and the downstream operators (including grouping) are uniformly lightweight (~100ms each). BitEngine thus reuses the standard operators for these stages, so the cost model the reviewer describes for fact-column grouping (one bitvector per group across all group-by columns) does not apply to Q10.

B.2 Q17

Q17 includes a notable correlated subquery: for every qualifying part, the average *l_quantity* across all of that part's lineitems

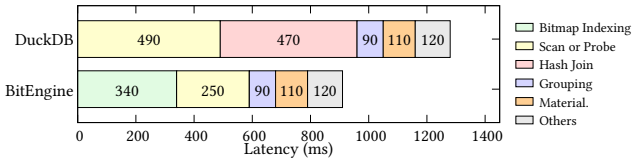

Figure 3: Per-stage latency breakdown of the two Q10 plans in Figure 2. The full LINEITEM scan (490ms) and hash-join build/probe (470ms) dominate 75% of DuckDB's latency. BitEngine replaces them with bitmap indexing (340ms) and a selective probe (250ms), 1.6× faster than the hash join in DuckDB. The downstream grouping (90ms) and materialization (110ms) stages are lightweight in both systems, confirming that the fact-table join, not the grouping, is the bottleneck for SPJA queries.

must be computed and then compared, per row, to *l_quantity* on LINEITEM.

DuckDB decorrelates the subquery, avoiding per-tuple re-evaluation of the subquery. Specifically, DuckDB scans LINEITEM and hash-joins it with the filtered PART to form the outer rows, while in a separate pipeline it scans LINEITEM for a second pass, joins with the distinct qualifying *p_partkey*s *D*, and groups by *p_partkey* to compute the per-key threshold (0.2×avg(*l_quantity*)). A delim-join then attaches each threshold back to the outer rows for the final filter and performs aggregation (*sum*). Our evaluation shows that this plan is suboptimal because it produces *two independent scans on LINEITEM*: one for the outer LINEITEM-PART join, and another for the LINEITEM-*D* join that computes the per-key average. On TPC-H SF=10, the two LINEITEM scans plus the two hash joins take 1,230ms, accounting for 95% of end-to-end latency.

Single-pass rewrite for Q17. Since *p_partkey* is the primary key of PART, the correlated subquery is semantically equivalent to a window aggregate partitioned by *l_partkey* over the post-join rows. Rather than modifying DuckDB's optimizer, we rewrite the query into the following single-pass form that scans LINEITEM only once.

```
SELECT sum(l_extendedprice) / 7.0 AS avg_yearly
FROM (
  SELECT l_extendedprice, l_quantity,
         avg(l_quantity) OVER (PARTITION BY l_partkey)
         AS avg_qty
  FROM lineitem JOIN part ON l_partkey = p_partkey
  WHERE  p_brand ='Brand#23' AND  p_container ='MED BOX'
) s
WHERE l_quantity < 0.2 * avg_qty;
```

On our rewritten Q17, DuckDB scans LINEITEM once, hash-joins it with the filtered PART, computes the window aggregate partitioned by *l_partkey* over the qualifying rows, and finally applies the outer filter and *sum*. This rewrite reduces DuckDB's Q17 latency from 1300ms (original plan) to 710ms.

BitEngine follows the same pipeline as the single-pass DuckDB but replaces the LINEITEM scan and hash join with bitmap indexing. As shown in Figure 4b, BitEngine first filters the dimension table PART (as in DuckDB), yielding ~0.1% of its rows as qualifying part keys. For each qualifying key, BitEngine fetches its bitvector from the *l_partkey* bitmap index and ORs it into a result bitvector that identifies the LINEITEM rows passing the join predicate. Since this bitvector is sparse, BitEngine converts it into an ID list (see §3.5) of only 61K row IDs. Thanks to efficient bitwise operations

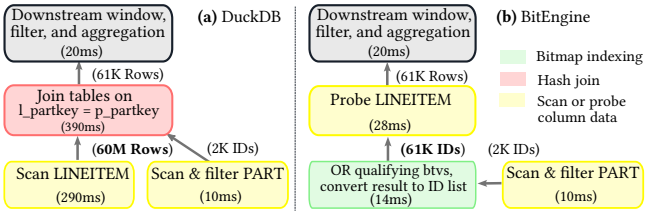

Figure 4: Query plans of our single-pass TPC-H Q17 (SF = 10) on (a) DuckDB and (b) BitEngine, with each operator annotated by its execution time and the number of rows it passes downstream. DuckDB scans the *entire* LINEITEM table (~60M rows) and passes all of them to the hash join with the filtered PART, whereas BitEngine ORs the *l_partkey* bitvectors of the qualifying part keys and probes LINEITEM for only the 61K matching rows (~0.1%). Both plans share the same downstream window-aggregate, filter, and aggregation operators. Figure 5 breaks down the resulting latencies.

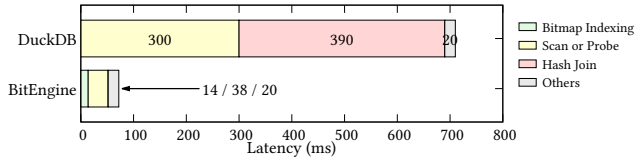

Figure 5: Per-stage latency breakdown of the two Q17 plans in Figure 4. The full LINEITEM scan (300ms) and hash-join build/probe (390ms) dominate 96% of DuckDB's latency. BitEngine replaces them with bitmap indexing (14ms) and a selective probe (38ms), yielding a 10× end-to-end speedup.

(§5), this indexing stage takes only 14ms [1]. BitEngine then probes LINEITEM only for these 61K rows (~0.1% of what DuckDB reads), cutting the probe to 38ms. The downstream operators run only on these 61K rows, so BitEngine reuses the standard operators.

Overall, **by replacing the full LINEITEM scan and hash join with bitmap indexing and a selective probe that reads only 0.1% of the tuples DuckDB does, BitEngine runs 10× faster than the best single-pass DuckDB**, as illustrated in Figure 5.

C JOB Queries

We extend our evaluation to the Join Order Benchmark (JOB) [1]. Unlike TPC-H's star schema, JOB is built on the real-world IMDB schema, with predominantly *fact-fact* joins. Its join keys are one to two orders of magnitude higher in cardinality than TPC-H's (e.g., *movie_id* has ~2.5M distinct values, *person_id* ~4.1M, and the *id* keys of *name*, *char_name*, and *title* reach 2.5–4.2M, as shown in Table 1). JOB therefore exercises both high-cardinality bitmaps (maintenance overhead) and fact-fact joins (cost-model robustness).

BitEngine Accelerates JOB because its join operator works for JOB's fact-fact joins seamlessly. For a fact-fact join, the standard hash-based join operator scans (and filters) two fact tables and then maintains a large hash table, which accounts for most of the latency of JOB queries. One key observation (demonstrated by both TPC-H and JOB) is that in analytical queries, joins are commonly

[1]Each *l_partkey* bitvector is sparse (~30 set bits in 60M rows), so the OR skips the all-zero runs and touches only the literal words, leaving ~30 cache misses as the dominant cost.

Table 1: BitEngine's bitmap maintenance overhead on JOB, measured along storage, build, and update cost. Cardinality (*C*) spans 3 to 4.2M, stressing high-cardinality bitmaps. Storage scales with the table length (*N*, the row count) rather than *C*, so the columns of the 36.2M-row *cast_info* table dominate. Even so, each bitmap index stays comparable to DuckDB's already-compressed base column (0.88× overall). Building all bitmaps is a one-time cost of 16.3 s, and each *UDI* stays sub-μs.

Column	*C*	*N* (rows)	Index (MB)	Base (MB)	Idx/ Base	Upd. (μs)	Build (s)
name.gender	3	4.2M	0.3	0.49	0.61	< 1	<0.01
cast_info.note	0.6M	36.2M	11.3	92.92	0.12	< 1	0.46
movie_kw.movie_id	2.5M	4.5M	2.4	3.10	0.77	< 1	0.40
cast_info.movie_id	2.5M	36.2M	134.3	98.94	1.36	< 1	3.94
title.id	2.5M	2.5M	9.6	5.28	1.85	< 1	2.86
char_name.id	3.1M	3.1M	11.9	6.52	1.83	< 1	2.91
cast_info.person_id	4.1M	36.2M	24.1	25.20	0.96	< 1	2.44
name.id	4.2M	4.2M	15.9	6.87	2.30	< 1	3.25
Total			**209.8**	**239.2**	**0.88**	~1	**16.27**

highly selective (see our response to R3.W4), which BitEngine exploits. Specifically, BitEngine evaluates each join through a bitmap on the FK column, parameterized by an input set K of qualifying key values. Whether K comes from a filtered dimension (as in TPC-H) or from an upstream filtered fact table (as in JOB) does not change the operator. For example, in JOB 6c, the *title–cast_info* join merges only 2 bitvectors of the *movie_id* index in *cast_info*, reducing 36.2M candidate rows to only 33, and the following *cast_info–name* join reduces qualifying rows to 20. BitEngine thus probes only those few rows and never materializes the 36.2M-row fact table, making it 7.1× faster than DuckDB (please see our response to R3.W4 for details). This confirms empirically that BitEngine's join operator generalizes beyond fact-dimension joins to arbitrary joins of large tables. Figure 6 reports end-to-end latency on eight representative JOB queries spanning (i) fact-fact *movie_id* chains of depth 2–5 (6a/b/c, 17a, 26a, 31a) and (ii) a varying selectivity on a fixed query shape (6a/b/c). Overall, BitEngine is faster on every query, by 1.6–8.5× (geomean 3.8×).

Maintenance Overhead. Table 1 reports the maintenance cost of every bitmap these queries use, at cardinalities from 3 to 4.2M. We report both the cardinality (C) and the table length (N) for each column, since both the base column size and the bitmap index scale with N rather than C. Overall, even at JOB's million-scale join keys, each bitmap index stays comparable to the already-compressed base column in DuckDB, at most 2.3× for *name.id* and 0.88× across the whole join graph (*Idx/Base*), because BitEngine's GE encoding with WAH compression (§3) keeps the per-value bitvectors compact even as cardinality grows. Updates and builds are equally lightweight, as in TPC-H, with each *UDI* remaining sub-μs and the entire join graph building once in 16.3 s.

C.1 Q6c example

We have evaluated BitEngine on JOB, which is dominated by fact-fact joins. For example, Q6c joins *cast_info* (36.2M rows) with *title* (2.5M), *name* (4.1M), and *movie_keyword* (2.5M).

BitEngine handles fact-fact joins without any modification. It converts a join operation to a foreign-key equality predicate evaluated

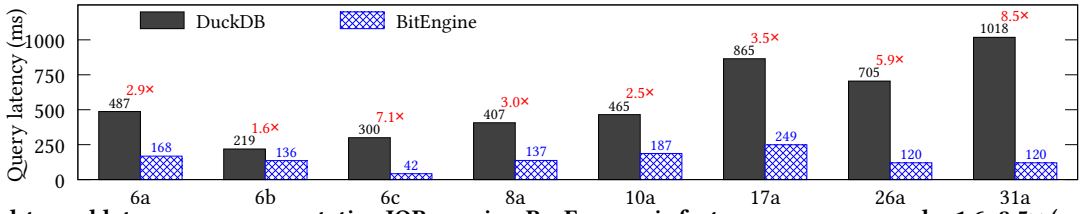

Figure 6: End-to-end latency on representative JOB queries. BitEngine is faster on every query, by 1.6–8.5× (geomean 3.8×), with the largest gains on the deep fact-fact join chains (26a, 31a). Rather than scanning the fact tables and building a large hash table per join as DuckDB does, BitEngine evaluates each join as an FK-column predicate answered with bitmaps, then probes only the few surviving rows, orders of magnitude fewer than scanning/filtering the fact table (e.g., 33 of 36.2M in Q6c, Figure 7).

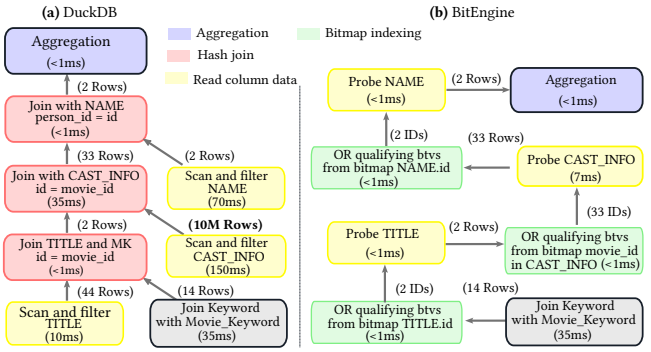

Figure 7: Query plan of JOB Q6c, a fact-fact join chain in which each join is highly selective (e.g., the *TITLE–CAST_INFO* join over 2.5M and 36M rows yields only 33 qualifying rows). DuckDB scans and filters each base table and maintains a hash table per join. BitEngine instead evaluates every join as a bitmap index probe on the foreign key, merging dozens of bitvectors and probing dozens of qualifying rows from the fact tables, achieving a 7.1× speedup.

through a bitmap on the FK column, parameterized by an input set K of qualifying key values. Whether K comes from a filtered dimension (e.g., most TPC-H queries) or from a previously filtered fact table (e.g., most JOB queries) does not change the operator. BitEngine's join cost is determined by the query selectivity from two angles: (1) the size of K, which determines how many bitvectors must be merged, and (2) the number of set bits in the result bitvector, which determines how many IDs pass to downstream operators (e.g., to probe the fact tables).

BitEngine accelerates analytical workloads like JOB because their joins are commonly highly selective. As an example, we illustrate the query plans of DuckDB and BitEngine for JOB Q6c, a chain of fact-fact joins. As shown in Figure 7, DuckDB runs it as a sequence of hash joins, so at every step it must scan and filter a base table and perform a hash join, which accounts for most of the query time (e.g., scanning *CAST_INFO* takes 150 ms and *NAME* 70 ms). BitEngine does neither. For each join, it ORs the qualifying per-key bitvectors on the foreign key, converts the resulting bitvector to a short ID list, and probes only the surviving rows for downstream operators. For example, the *TITLE–CAST_INFO* join merges only 2 bitvectors from the bitmap on *CAST_INFO.movie_id*, and the result bitvector has only 33 set bits, so BitEngine probes just 33 rows of *CAST_INFO* in 7 ms. Overall, by replacing every base-table scan and hash-table build with bitmap indexing plus a tiny probe, BitEngine finishes Q6c in 42 ms, 7.1× faster than DuckDB.

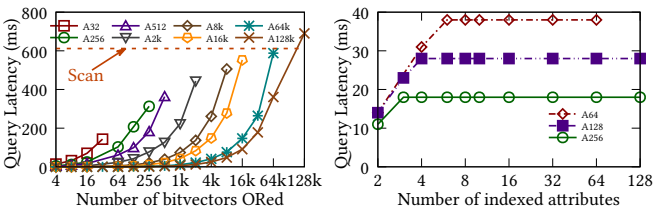

Figure 8: (a) ORing 64K or fewer bitmap bitvectors is faster than scanning the corresponding column data, and (b) the cost of ANDing bitvectors from different bitmap instances increases for a few ANDs and then flattens.

D Efficient Bitvector Merges

BitEngine's Join operator needs to retrieve a large number of bitvectors and perform extensive bitwise operations over them when applied to high-cardinality attributes. For example, in our simplified TPC-H Q5 example, when SF = 10, the bitmap instance on the *l_orderkey* attribute of the LINEITEM table contains ~15 million bitvectors, and our Join operator involves ~457K logical ORs, which was previously prohibitive. We address this inefficiency effectively from the following perspective.

BitEngine's Merge Mechanism leverages the observation that, for high-cardinality attributes using the EE or GE schemes, each bitvector contains a few 1s (~4 1s in this bitmap instance) and is highly compressible (~16 bytes/bitvector). This allows the merge mechanism to skip most portions of the bitvector during merging. To demonstrate this, we evaluate the cost of merging a group of bitvectors. Figure 8a presents the execution time for bitwise ORs across a group of bitvectors while varying the cardinality of the indexed attributes from 32 to 128K (as indicated by the legend suffixes). For comparison, we also include execution time for scanning the column using a two-sided predicate. We observe that ORing 64K bitvectors to retrieve the result bitvector is faster than scanning the base data. The main reasons are as follows. (1) With higher cardinality, the bitvectors have lower bit density and are more compressible, allowing us to skip most portions of the intermediate bitvector (*IB*) during merging. (2) As 1s accumulate, *IB* becomes less compressible and is maintained as decompressed, significantly reducing the cost of merging 1s into it.

Multi-Column Predicates. Realistic queries often involve logical ANDs over bitvectors generated by multi-column predicates and star joins. For example, in TPC-H Q5, the final result bitvector for *l_price* is computed by ANDing the join result bitvectors from *LINEITEM-ORDERS* and *LINEITEM-SUPPLIER*. Figure 8b reports the execution time for logical ANDs between bitvectors from different bitmap instances, with cardinality varying from 64 to 256. Our evaluation

shows that the query latency plateaus after a few ANDs because *IB*'s bit density decreases dramatically and becomes highly compressible, enabling the skipping of most portions in the subsequent ANDs, making ANDing join result bitvectors efficient.

References

[1] Viktor Leis, Andrey Gubichev, Atanas Mirchev, Peter A Boncz, Alfons Kemper, and Thomas Neumann. 2015. How Good Are Query Optimizers, Really? *Proceedings of the VLDB Endowment* 9, 3 (2015), 204–215. http://www.vldb.org/pvldb/vol9/p204-leis.pdf